



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Priority Queues - Introduction



Agenda & Reading

- ▶ **Agenda**
 - ▶ Introduction to Priority Queues
- ▶ **Reading**
 - ▶ Online Textbook: <https://onlinetextbook.auckland.ac.nz/20-priority-queues-and-heaps/>





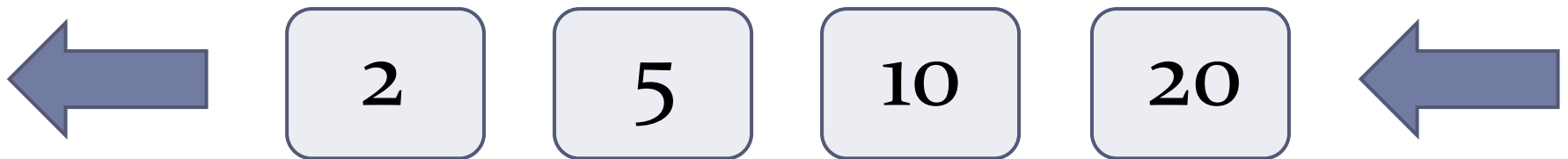
Priority Queues

- ▶ A **priority queue** is a variation of a queue
 - ▶ with a queue, items are dequeued (removed) in the *same order* they are added
 - ▶ with a **priority queue**, each item has an associated priority and **items are dequeued** (removed) **based on their priority**
 - ▶ the highest priority items are at the front of the queue and the lowest priority items are at the rear
 - ▶ it is therefore possible for an item to be enqueued (added) and move straight to the front of the queue (if it has the highest priority amongst all items)



Priority Queues

- ▶ Abstract Datatype (ADT) Operations:
 - ▶ insert()
 - ▶ delete_minimum()
 - ▶ peek()





Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list		
Sorted array / Python list		
Unsorted linked list		
Sorted linked list		
Hashtable		
Binary search tree		



Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list		
Unsorted linked list		
Sorted linked list		
Hashtable		
Binary search tree		



Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list	$O(n)$	$O(1)$
Unsorted linked list		
Sorted linked list		
Hashtable		
Binary search tree		



Example

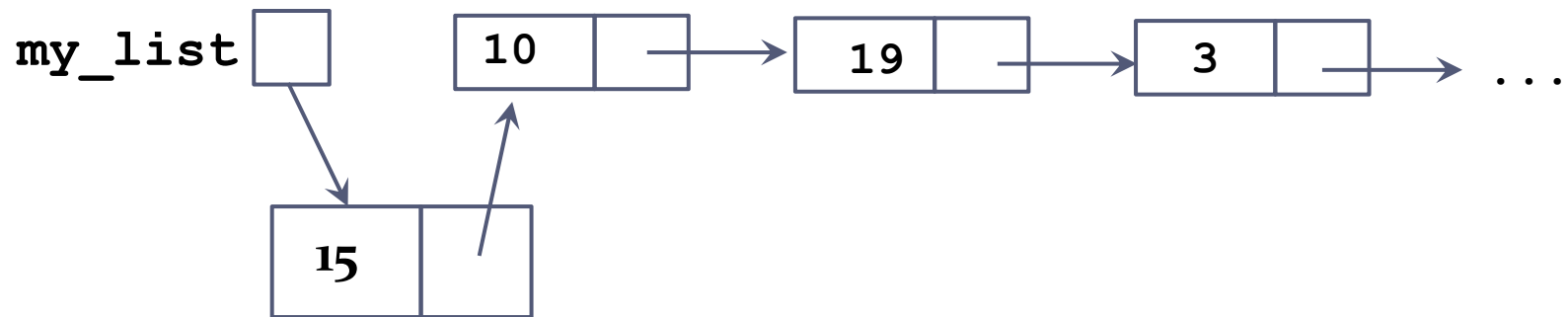
- ▶ Inserting a value into an unsorted linked list:





Example

- ▶ Inserting a value into an unsorted linked list:





Priority Queues

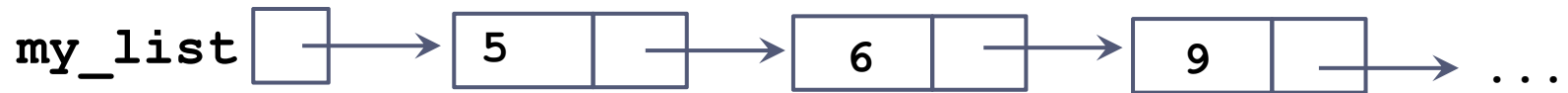
- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list	$O(n)$	$O(1)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list		
Hashtable		
Binary search tree		



Example

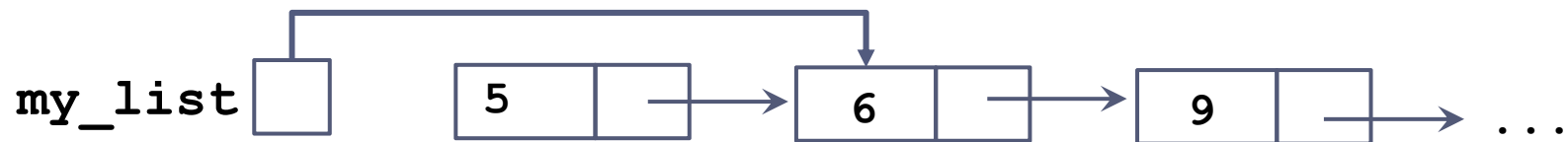
- ▶ Deleting the minimum value from a *sorted* linked list:





Example

- ▶ Deleting the minimum value from a *sorted* linked list:





Example

- ▶ Deleting the minimum value from a *sorted* linked list:





Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list	$O(n)$	$O(1)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$
Hashtable		
Binary search tree		



Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list	$O(n)$	$O(1)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$
Hashtable	$O(1)$	$O(n)$
Binary search tree		



Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently

Data structure	Insert item	Remove min/max
Unsorted array / Python list	$O(1)$	$O(n)$
Sorted array / Python list	$O(n)$	$O(1)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$
Hashtable	$O(1)$	$O(n)$
Binary search tree	$O(\log n)$	$O(\log n)$

(if balanced)



Priority Queues

- ▶ How can we implement a priority queue?
 - ▶ it is equivalent to implementing a collection from which we can find and remove the **minimum** (or maximum) element efficiently
- ▶ Implementation of ADT operations required:
 - ▶ Create new priority queue
 - ▶ **insert()**: Add an item (typically a priority / value pair)
 - ▶ **delete_minimum()**: Remove item with highest priority (encoded with minimum)
 - ▶ **peek()**: Report the item with the highest priority, without removing it!